

Ereko African Market

Production Product Requirements Document (PRD) — v2.0

Document Type: Enterprise Product Requirements Document **Product:** Ereko African Market — Premium African Grocery, Specialty Food & Cargo Platform **Region (Phase 1):** United Kingdom **Status:** Master Build Specification — Rebuild from Beta **Engineering Stack (canonical):** Next.js 15 (App Router) · TypeScript · Supabase (Postgres + Auth + Storage + Realtime + Edge Functions) · Stripe (Payments + Radar) · Resend/Postmark (Transactional Email) · Twilio/MessageBird (SMS + WhatsApp) · Upstash (Redis/Queues) · Algolia or Meilisearch (Search) · Cloudflare (CDN + WAF + Images) · Vercel (Hosting) · Sentry + PostHog + Grafana (Observability)

0. Document Conventions

To remove ambiguity across engineering, design, QA, and operations, this PRD uses RFC 2119 keywords:

- **MUST / SHALL** — Hard requirement. Failure = bug.
- **SHOULD** — Strong recommendation. Deviations require an architectural decision record (ADR).
- **MAY** — Optional, at team discretion.
- **OUT OF SCOPE (v1)** — Explicitly deferred but architecturally accommodated.

Every functional section concludes with **Acceptance Criteria** written as Gherkin-style scenarios so QA can map them 1:1 to automated tests.

1. Executive Summary

Ereko African Market is a premium, enterprise-grade commerce and commerce-operations platform serving the diaspora and multicultural UK market with authentic African groceries, specialty foods (fresh, dry, frozen, and chilled), household goods, beauty products, and a parallel **cargo/freight forwarding service** for customers shipping goods to West African destinations.

The product is being rebuilt from a v1 beta with three non-negotiable objectives:

1. **Security and trust as first-class concerns** — PCI DSS v4.0.1 alignment, OWASP ASVS Level 2 baseline, GDPR/UK-DPA 2018 by design, Stripe Radar fraud screening, server-authoritative pricing and inventory, and full audit logging.

- 2. **Operational depth** — A platform that does not stop at the "buy button." Multi-warehouse inventory, pick-pack-ship workflows, substitutions, returns, refunds, dispute handling, finance reconciliation, and a workflow engine for staff approvals.
- 3. **Customer experience at parity with Tier-1 grocers** — WCAG 2.2 AA accessibility, Core Web Vitals "Good" thresholds, AI-personalized merchandising, delivery slot booking, BNPL/Apple Pay/Google Pay, loyalty with gamification, and a localized identity that respects the cultural specificity of African cuisine.

This PRD specifies the entire surface area: data model, API contracts, security posture, frontend architecture, backend services, integrations, QA strategy, observability, rollout plan, and the operational runbooks required to take the platform from rebuild kickoff to production scale.

2. Strategic Positioning

2.1 Market Context

The UK ethnic food market is a high-growth segment of UK grocery, driven by diaspora demand, multicultural household formation, and a maturing consumer expectation that *specialty* online grocery should feel as polished as mainstream supermarket e-commerce. Competitors like Niyis, WosWosi, Everything African, and a long tail of independent shops have validated demand but most lag in checkout UX, accessibility, inventory accuracy, and post-purchase service quality.

2.2 Brand Pillars

Pillar	Translation into product
Authenticity	Cultural product taxonomy (e.g., <i>egusi</i> , <i>ogbono</i> , <i>crayfish</i> , <i>plantain flour</i> , <i>kpomo</i>), regional origin badges (Nigeria, Ghana, Cameroon, Ivory Coast, Sierra Leone, etc.), and authentic visual identity.
Reliability	Real-time stock, accurate ETAs, transparent substitution policy, proactive comms, and a returns experience that does not feel adversarial.
Premium	Editorial product photography, recipe-led merchandising, white-glove customer service, and packaging that protects fresh/frozen integrity.
Community	Loyalty + referrals, recipe content, festival/holiday campaigns (Christmas hampers, Ramadan, Eid, Independence Day promotions), and a "request a product" channel.

2.3 Business Outcomes (KPIs)

Outcome	Metric	Year-1 target
Conversion	Sessions → Paid orders	≥ 3.0% overall; ≥ 4.5% authenticated returning customers
Repeat purchase		≥ 35%

Outcome	Metric	Year-1 target
	Customers placing ≥ 2 orders within 90 days	
Average order value (AOV)	Mean order value (ex-VAT, ex-delivery)	£55+
Cart abandonment	Abandoned checkouts $\div$ initiated checkouts	$\leq 65\%$ (industry baseline $\sim 70\%$); recovery flow target $\geq 8\%$ recovery
Order accuracy	Orders fulfilled without substitution complaint	$\geq 97\%$
First-contact resolution (support)	Cases resolved in first reply	$\geq 70\%$
NPS	Post-delivery NPS	≥ 50
Core Web Vitals	% of mobile sessions passing all three CWV thresholds	$\geq 90\%$

3. Product Principles

These principles arbitrate every design and engineering decision.

1. **Server-authoritative commerce.** Prices, stock, taxes, shipping, discounts, and loyalty redemptions are calculated and re-validated on the server at every state transition. The client is a *renderer*, never an authority.
2. **Idempotency everywhere money moves.** Every checkout, payment confirmation, refund, and webhook handler is idempotent via Stripe idempotency keys + database-stored idempotency tokens.
3. **Accessibility is a hard gate.** No feature ships without keyboard, screen reader, and color-contrast verification. WCAG 2.2 AA is the floor, not the ceiling.
4. **Performance is a feature.** Every surface has a performance budget enforced in CI (Lighthouse CI + bundle-size checks). Storefront pages: LCP ≤ 2.0 s on mobile 4G, INP ≤ 200 ms, CLS ≤ 0.1 .
5. **Audit by default.** Every privileged action writes to an immutable audit log. The log is queryable and exportable.
6. **No hardcoded business rules.** Tax rates, shipping zones, promotions, and loyalty rules live in configuration tables. Code interprets configuration; it does not embed it.
7. **Mobile-first storefront, desktop-efficient admin.** Two distinct UX philosophies in one codebase.
8. **Observable, recoverable, reversible.** Every operation is logged, every state transition has a rollback path, every job is retryable with backoff.
9. **Security by design.** Threat modeling occurs before implementation. Secrets never reach the client bundle. RLS at the database is the last line of defense, not the first.
10. **Configurable, not code-dependent, for tax, shipping, and promotions.** Operators change behavior via admin UI; engineers change behavior via code only when capability is missing.

4. Scope

4.1 In Scope (v1.0 — Public Launch)

- Customer-facing storefront (web, responsive PWA).
- Guest and authenticated checkout.
- Customer account portal (orders, addresses, payment methods, loyalty, wishlist, returns, communication preferences).
- Multi-warehouse inventory and fulfillment.
- Order, shipment, return, and refund workflows.
- Promotions and coupon engine.
- Loyalty program (points, tiers, referrals).
- Admin console with RBAC, workflows, and audit logs.
- Warehouse operations dashboard (receiving, picking, packing, cycle counts, transfers).
- Shipping zones, carrier integration, delivery slot booking.
- Tax engine (UK VAT initially, multi-region ready).
- Payment processing (Stripe — cards, Apple Pay, Google Pay, Klarna, Clearpay).
- Notifications (transactional email, SMS, WhatsApp; marketing email via ESP).
- Search and merchandising controls.
- Recipe content hub (CMS).
- Cargo/service inquiry intake and case management.
- Analytics, reporting, and exports.
- Integrations with payments, email, SMS/WhatsApp, shipping carriers, accounting export.

4.2 In Scope (v1.1–v1.3 Follow-ons)

- Subscription/repeat-order box.
- Gift cards.
- Advanced segmentation engine.
- B2B/wholesale portal (light).
- Native PWA install prompts and push notifications.

4.3 Out of Scope for v1

- Native iOS/Android apps (PWA covers v1).
- Marketplace seller portal.
- Full ERP replacement (accounting export only).
- In-store POS integration.
- Drop-shipping and 3PL integration beyond carrier APIs.
- Auto-translation/full i18n beyond English (architecture is i18n-ready).

5. Roles, Permissions, and RBAC

The platform implements role-based access control with **resource-level scopes**. A role is a named bundle of permissions; a permission is `resource:action:scope` (e.g., `orders:refund:own_warehouse`).

5.1 Role Matrix

Role	Description	Sample Permissions
Guest	Anonymous visitor	<code>catalog:read</code> , <code>cart:write:session</code> , <code>checkout:start</code>
Customer	Registered shopper	All Guest + <code>account:*:self</code> , <code>orders:read:self</code> , <code>loyalty:redeem:self</code> , <code>returns:create:self</code>
Support Agent (Tier 1)	Front-line CX	<code>customers:read</code> , <code>orders:read</code> , <code>orders:annotate</code> , <code>returns:create</code> , <code>refunds:request</code> (requires approval)
Support Lead (Tier 2)	Senior CX	All Tier 1 + <code>refunds:approve:≤£100</code> , <code>orders:edit:limited</code> , <code>customers:edit</code>
Warehouse Operator	Pick/pack/receive	<code>warehouse:tasks:*:own_warehouse</code> , <code>stock:move:own_warehouse</code> , <code>shipments:pack</code>
Warehouse Manager	Warehouse ops lead	All Operator + <code>stock:adjust:own_warehouse</code> , <code>transfers:approve</code> , <code>cycle_counts:approve</code>
Logistics Manager	Carrier + routing	<code>shipments:*</code> , <code>carriers:configure</code> , <code>shipping_zones:configure</code>
Catalog Manager	Products/content	<code>catalog:*</code> , <code>media:*</code> , <code>categories:*</code> , <code>seo:*</code>
Marketing Manager	Campaigns/loyalty	<code>promotions:*</code> , <code>loyalty:configure</code> , <code>segments:*</code> , <code>campaigns:*</code>
Finance Manager	Reconciliation	<code>refunds:approve:any</code> , <code>payments:read</code> , <code>exports:finance:*</code> , <code>settlements:read</code>
Admin	Operations lead	Broad operational access, no system config
Super Admin	System owner	Everything, including roles, integrations, audit log, feature flags

5.2 RBAC Implementation Requirements

- Roles MUST be database-driven, not hardcoded.
- Permissions MUST be checked at three layers: (1) UI route guard, (2) API/server action middleware, (3) Postgres Row-Level Security (RLS) on Supabase.

- Sensitive actions (refunds > threshold, bulk price changes, role assignments, integration secret rotation) **MUST** require **step-up auth** (re-enter password or MFA challenge) regardless of session age.
 - The system **MUST** support **deputization** (temporary role grant with expiry and reason code), logged to audit trail.
 - Service accounts (for integrations) **MUST** have their own roles, separate from human users, with strictly minimal scopes.
-

6. End-to-End User Journeys

6.1 Discover → Buy (Customer)

1. Customer lands on homepage; sees seasonal hero (e.g., "Christmas Hamper Pre-Orders Open"), curated rails (Bestsellers, New In, Festive, Frozen Essentials), and personalized "Recommended for You" if returning.
2. Searches "egusi seeds"; instant-search suggests products, categories, and recipes.
3. Filters by *origin: Nigeria, in stock, dietary: vegan*. Sorts by *popularity*.
4. Product detail page (PDP) shows hero gallery, origin badge, ingredients, allergen panel, weight/units, dietary tags, recipe pairings, delivery ETA based on postcode (entered or geo-inferred), stock indicator, points earned, "Save for Later," and "Add to Cart."
5. Adds 4 items, opens cart, applies code **FAMILY10**, sees VAT-inclusive total, loyalty points earnable, and free-delivery progress bar.
6. Proceeds to checkout; address autofilled from previous order (or postcode lookup). Selects delivery slot (Sat 09:00–13:00). Chooses Apple Pay. Completes 3-D Secure if challenged. Sees confirmation with order number, tracking link, and recipe suggestions for the items purchased.

6.2 Order → Doorstep (Operations + Customer)

1. Order is created in **PENDING_PAYMENT**. Payment Intent confirmed → webhook transitions to **PAID** and **ALLOCATED** to the optimal warehouse based on stock and postcode.
2. Picker receives task on warehouse dashboard; picks items with barcode confirmation. Out-of-stock item triggers substitution workflow: system offers configured alternates; customer approval policy (auto-accept, ask, or never) is honored.
3. Packer verifies and packs; cold-chain items go into insulated packaging with cold packs; system prints carrier label and invoice. Status → **PACKED** → **SHIPPED** (with tracking number written back).
4. Courier scans → carrier webhook → status updates → customer receives SMS/email with live tracking.
5. On delivery, customer receives "How was it?" survey 24 hours later and a recipe email tied to their purchased ingredients.

6.3 Return & Refund (Customer + Support + Finance)

1. Customer opens "My Orders," selects an order, clicks "Return or Report an Issue."

2. Selects reason (Damaged, Wrong Item, Quality Issue, Missing Item, Changed Mind), uploads photo evidence.
3. Return Merchandise Authorization (RMA) created in **PENDING_REVIEW**. Routed to Support queue.
4. Support reviews. If approved, refund proposed; if amount $\leq$ Tier-2 threshold, Support Lead approves; if $>$ threshold, Finance approves.
5. Refund executed via Stripe Refunds API (idempotent); loyalty points adjusted; stock returned to inventory if applicable; customer notified.

6.4 Cargo/Freight Inquiry

1. Customer fills cargo inquiry form (origin, destination, items, weight/volume estimate, urgency, contact info).
2. Inquiry creates a **case** in Support queue; SLA timer starts.
3. Cargo desk responds with quote and booking link; case progresses through **QUOTED**, **BOOKED**, **IN_TRANSIT**, **DELIVERED**, **CLOSED**.

7. Functional Requirements

7.1 Catalog & Product Information Management (PIM-Lite)

7.1.1 Capabilities

- Products **MUST** support: title, slug, short description, long description (rich text), brand, **origin country**, weight (numeric + unit), dimensions, ingredients (free text + structured allergens), nutritional information (per 100g), dietary tags (vegan, vegetarian, halal, gluten-free, lactose-free, kosher, organic), storage type (ambient, chilled, frozen), shelf-life policy, and SKU.
- Variants **MUST** be supported (e.g., 1kg vs 5kg, 250ml vs 1L), each with its own SKU, price, stock, and barcode (EAN/UPC).
- Bundles (kits, hampers) **MUST** be supported as a parent SKU referencing child SKUs with quantities; pricing rules allow fixed price or sum-of-children-with-discount.
- Products **MUST** support category multi-assignment, tags, and collections (curated lists with optional auto-rules).
- Media: multiple images per product, 360° spin (optional), video URL, alt text required. Images served via Cloudflare Images / Supabase Storage with on-the-fly resizing.
- SEO fields: meta title, meta description, canonical URL, OG image, structured data (Product, Offer, AggregateRating, Recipe).
- **Cultural metadata** unique to Ereko: regional cuisine (Nigerian, Ghanaian, Cameroonian, Ivorian, Sierra Leonean, etc.), local names (e.g., *crayfish* / *ede*), traditional uses ("Used in soups, stews, and jollof rice"), pairings (other SKUs).

7.1.2 PIM Workflows

- Draft → Review → Published lifecycle with optional approval gate.
- Bulk import via CSV with column mapping, validation report, and dry-run mode.
- Image bulk upload with automatic alt-text suggestions (AI assist, manually overridable).
- Versioning: changes to published products create a new version; previous version retained for rollback.
- Scheduled publishing/unpublishing.

7.1.3 Acceptance Criteria

Given a Catalog Manager creates a new product with all required fields,
When they click "Publish",
Then the product **MUST** appear on storefront within 60 seconds,
And appear in search results within the same window,
And generate a sitemap entry.

Given a product has a published version,
When the Catalog Manager modifies it and saves,
Then a new draft version **MUST** be created without affecting the live version,
Until "Publish" is clicked.

Given a CSV import of 5,000 SKUs with 12 validation errors,
When the import is run in dry-run mode,
Then the system **MUST** report all 12 errors with row numbers and field names,
And persist no changes.

7.2 Search, Browse, and Merchandising

- The storefront **MUST** support **instant search** (typeahead) with results within 100 ms (p95) on warm cache. Vendor: Algolia or Meilisearch (self-hosted). Decision recorded in ADR-001.
- Search **MUST** support typo tolerance, synonyms (e.g., "crayfish" ↔ "ede" ↔ "dried shrimp"), language-aware tokenization, and cultural-term recognition.
- Faceted filtering **MUST** be supported on: category, brand, origin, dietary tags, price range, storage type, in-stock status, promotions, and warehouse availability (when delivery postcode is set).
- Sort options: relevance, popularity (rolling 30-day), bestsellers (lifetime), newest, price (asc/desc), discount %.
- Merchandising controls (Marketing Manager): pin products to top of category, hide products, boost by query, banner overlays, A/B test landing variants.
- **Search analytics**: zero-result queries, click-through rate per result position, conversion per query — surfaced in Marketing dashboard.

7.3 Cart

- Cart **MUST** persist per-user on the server (PostgreSQL) for authenticated users; for guests, in `localStorage` with secure HMAC signature, then merged on login (server-side merge resolves conflicts by max-quantity).
- Cart **MUST** support: line items with variant, quantity, promo codes, gift-wrapping notes, delivery slot selection, and reward redemption.

- Cart MUST be **price-revalidated on every render and at checkout entry**. If prices have changed, customer is notified with diff before proceeding.
- Out-of-stock or low-stock items MUST be highlighted with clear messaging and CTAs (substitute, remove, save for later).
- Free-delivery progress bar MUST be visible when configured threshold is approachable.

7.4 Checkout

7.4.1 Flow

- One-page checkout (single scrollable page) with progressive disclosure of payment after contact + delivery details are valid.
- Steps: Contact → Delivery (address + slot) → Payment → Review.
- Guest checkout MUST be supported; "Create account at end" MUST be offered.
- Address: postcode-first lookup (Loqate/getAddress.io), manual override allowed, address validation against carrier serviceability before proceeding to payment.
- Delivery options: standard, express, click-and-collect (if any warehouse opts in), slot booking (date + time window).
- Payment methods (v1, UK):
 - Cards (Visa, Mastercard, Amex) via Stripe
 - Apple Pay
 - Google Pay
 - Klarna (pay-in-3, pay-in-30)
 - Clearpay (formerly Afterpay UK)
 - PayPal (optional, deferred to v1.1)
- 3-D Secure 2 MUST be enforced for all card transactions (Stripe Radar will trigger when risk warrants).
- Order is created **before** payment confirmation in `PENDING_PAYMENT` state. Stripe `payment_intent.succeeded` webhook transitions to `PAID`.
- Stock MUST be **reserved (soft-allocated)** the moment checkout begins (configurable 15-minute hold). Reservation expires on abandonment.

7.4.2 Idempotency & Reliability

- A unique `checkout_attempt_id` MUST accompany every payment intent. Replays MUST return the same payment intent rather than create duplicates.
- Webhook handlers MUST be idempotent: handlers check an `idempotency_events` table before applying state changes.
- Webhook signatures MUST be verified using Stripe's signing secret on every request.
- Failed payments must transition cart back to "Open" with the failure reason surfaced to the user; reserved stock is released after grace period.

7.4.3 Acceptance Criteria

Given a customer submits payment with an idempotency key,
When the same key is submitted again within 24 hours due to client retry,
Then no duplicate charge MUST occur,
And the original payment result MUST be returned.

Given the Stripe webhook fails to deliver an event,
When Stripe retries with the same event ID,
Then the system MUST detect duplicate processing and short-circuit safely,
And the audit log MUST record one and only one state transition.

Given checkout reserves stock for 15 minutes,
When 16 minutes pass without payment confirmation,
Then reserved stock MUST be released,
And the cart MUST remain accessible but flagged for re-validation.

7.5 Orders & Fulfillment

7.5.1 Order State Machine

States: `PENDING_PAYMENT` → `PAID` → `ALLOCATED` → `PICKING` → `PACKED` → `SHIPPED` → `OUT_FOR_DELIVERY` → `DELIVERED` → (optional) `RETURN_REQUESTED` → `RETURNED` → `REFUNDED`.

Side-states: `ON_HOLD`, `CANCELLED`, `PARTIALLY_SHIPPED`, `PARTIALLY_REFUNDED`, `DISPUTED`.

Every transition MUST: - Be triggered by a typed event with a payload schema. - Write to `order_events` (immutable). - Emit a domain event for downstream consumers (notifications, analytics, accounting).

7.5.2 Allocation Rules

- Default: nearest warehouse with full stock.
- Fallback 1: split shipment across warehouses if no single warehouse has full stock.
- Fallback 2: backorder if backorder is enabled for the SKU (must be flagged on PDP).
- Operator override MUST be available; overrides write a reason code to audit log.

7.5.3 Substitutions

- Per-SKU substitution policy (admin-configurable): one of `NEVER`, `AUTO_ACCEPT_CONFIGURED_ALTERNATE`, `ASK_CUSTOMER_AT_CHECKOUT`, `ASK_CUSTOMER_AT_PICK`.
- Per-customer override at checkout: "Substitute if needed" / "No substitutes" toggle.
- Substitution events MUST be logged and shown on order detail to both customer and ops.

7.5.4 Partial Fulfillment & Backorders

- Orders MAY ship in multiple shipments. Each shipment has its own tracking, status, and carrier label.
- Backordered items MUST display an updated ETA on order detail; customers MUST be able to cancel a backordered line.

7.5.5 Acceptance Criteria

Given an order is in PACKED state,
When a Warehouse Operator attempts to edit line items,
Then the system MUST require a privileged override with a reason code,
And the override MUST be logged to audit trail.

Given a paid order cannot be allocated to a single warehouse,
When allocation runs,
Then the order MUST be split across the minimum number of warehouses,
And each split MUST receive its own shipment record and tracking number.

7.6 Returns, Refunds & Disputes (RMA)

- Returns MUST be initiable from customer portal up to N days post-delivery (N configurable, default 14, longer for fresh/perishable returns disallowed).
- Reason codes MUST be standardized; photo upload required for damage/quality claims.
- Refund types: full, partial, store credit, loyalty points top-up.
- Refunds MUST go through approval workflow with thresholds per role.
- Disputes (Stripe chargebacks) MUST create a **dispute** record and notify Finance; dispute response evidence (delivery proof, customer comms, receipts) MUST be assemblable from the order detail in one click.

7.7 Loyalty Program — "Ereko Family"

7.7.1 Mechanics

- Points earned per £ spent (configurable, default 1 point per £1 ex-VAT, ex-delivery).
- Tiers: **Member** (0+), **Family** (£500 lifetime), **Elder** (£2,000 lifetime), **Royalty** (£5,000 lifetime). Tier benefits include points multipliers, free delivery thresholds, early access to drops, and exclusive bundles.
- Redemption: points convertible at checkout (configurable, default 100 points = £1 off, with max % of basket).
- Referrals: shareable code; referrer gets £X store credit when referee completes first qualifying order.
- Campaign bonuses: 2x points weekends, category boosts, birthday bonuses.

7.7.2 Gamification (v1.2)

- Streak rewards (consecutive months with an order).
- Quest system ("Try 3 new spices this month for 200 bonus points").
- Visible progress bars and tier-progress meters in account portal.

7.7.3 Acceptance Criteria

Given a customer has 250 loyalty points,
 When they redeem 200 points at checkout,
 Then £2.00 MUST be deducted from order total,
 And points balance MUST decrement atomically with order capture,
 And on order cancellation, points MUST be refunded.

Given a referral code is used by a new customer,
 When that customer completes a qualifying first order,
 Then the referrer MUST receive credit within 24 hours,
 And both parties MUST be notified.

7.8 Promotions & Coupon Engine

- Promotion types: percentage off, fixed amount off, BOGO, bundle price, free shipping, free gift with purchase, tiered discount (spend £X, get Y% off).
- Eligibility rules: customer segment, product/category, minimum spend, first-order, single-use vs multi-use, per-customer usage limit, total usage limit, date window, warehouse/region, day-of-week.
- Stacking rules: explicit `stackable: true/false`, priority order.
- Coupon code generation: bulk generation, unique codes for influencer/partner campaigns, vanity codes for marketing.
- Anti-abuse: rate limit per IP/customer, fraud-screen flags on suspicious redemption velocity.

7.9 Notifications

Channel	Use cases
Email (transactional)	Order confirmation, payment receipt, shipping notification, delivery confirmation, return updates, password reset, account verification
Email (marketing)	Newsletter, abandoned cart, win-back, post-purchase recipe, loyalty status changes
SMS	Out-for-delivery, slot reminders, OTP for sensitive actions
WhatsApp	Order updates (opt-in), customer service replies
In-app/Web push	(v1.1) PWA push for slot reminders and back-in-stock

Requirements: - All transactional templates MUST be MJML-based, responsive, and previewable per locale. - Customer notification preferences MUST be granular (per channel × per event type). - All marketing comms MUST honor unsubscribe and double-opt-in where mandated. - Webhook delivery to notification service MUST be queued with retries; failures alert ops.

7.9.1 Abandoned Cart & Lifecycle Flows

- Cart abandonment flow: t+1h email, t+24h email with offer, t+72h final reminder. Configurable per segment.
- Browse abandonment, wishlist-back-in-stock, post-first-purchase onboarding, churn win-back at 60/90/120 days, birthday and anniversary flows.

7.10 Search, Recipes & Content Hub

A first-class content hub differentiates Ereko from commodity grocers:

- Recipes (CMS-managed): title, hero image, ingredient list with **linked SKUs** (one-click "Add all ingredients to cart"), step-by-step instructions, cook time, servings, video embed, related recipes.
- Editorial pages: festival guides ("Christmas in Lagos"), how-to articles ("How to choose ripe plantain"), origin stories.
- All content is indexed by search and supports structured data (Recipe schema for Google).

7.11 Customer Account Portal

- Dashboard: tier badge, points balance, recent orders, recommended for you, "your usual" reorder list.
- Orders: list with filter (status, date, value), order detail with tracking, invoice download, "Reorder" button, "Return" button.
- Addresses: add/edit/delete, default shipping/billing, postcode-validated.
- Payment methods: tokenized via Stripe; never stored raw; default selection.
- Wishlist + Save for Later: separate from cart, shareable via private URL, configurable public profile.
- Subscriptions (v1.1): manage recurring boxes.
- Notification preferences: per channel × event type.
- Privacy & data: GDPR self-service — data export (JSON), account deletion (with grace period), consent history.

7.12 Admin Console

The admin is a dense, fast, keyboard-friendly application.

Top-level modules: - **Dashboard** — Today's orders, revenue, top SKUs, alerts (low stock, webhook failures, dispute count, refund queue). - **Orders** — Inbox-style queue, bulk actions, filters, search, save-views. - **Products** — Grid view, bulk edit, draft/published toggle, image bulk upload. - **Categories & Collections** — Hierarchy editor, drag-and-drop. - **Inventory** — Per-warehouse view, low-stock alerts, transfers. - **Warehouses** — Locations, operators, working hours, capacity. - **Customers** — Search, CLV, segments, order history, support history. - **Promotions** — Builder UI with eligibility preview. - **Loyalty** — Tier rules, points config, redemption settings, campaign builder. - **Returns & Refunds** — Queue, approvals, refund execution. - **Shipping & Tax** — Zones, carriers, tax rules, free-shipping rules. - **Content** — Pages, recipes, banners, navigation. - **Notifications** — Templates, segments, schedule, deliverability dashboard. - **Reports** — Sales, inventory, fulfillment, customer cohorts. - **Users & Roles** — Staff management, role assignment, audit log viewer. - **Settings** — General, branding, integrations, feature flags, API keys. - **Audit Log** — Filterable, exportable, immutable.

7.12.1 Admin UX Principles

- Every list **MUST** support filter, sort, save-view, bulk action, and CSV export.
- Every detail page **MUST** have a sidebar of related actions (refund, annotate, cancel, etc.).
- Every destructive action **MUST** require typed confirmation ("type the order number to cancel").
- Inline editing where possible; full-form editing for complex changes.

7.13 Warehouse Operations

- **Receiving** — PO-driven; barcode scan in; reconciliation against expected quantities; damaged-on-arrival capture.
- **Putaway** — Bin-location assignment; supports zone strategy.
- **Picking** — Wave picking or single-order picking; mobile-optimized UI; barcode scan confirms SKU.
- **Substitution at pick** — If picker can't locate item, suggested alternates appear; customer policy honored.
- **Packing** — Verifies basket against order; prints label and packing slip; captures cold-chain checklist for chilled/frozen.
- **Shipping** — Hands off to carrier; manifests generated end-of-day.
- **Cycle counts** — Scheduled, ad-hoc, or trigger-based (after damaged-stock event).
- **Transfers** — Initiate, approve, in-transit, received, with full audit.
- **Damaged stock** — Quarantine bin; reason codes; write-off workflow with finance approval.

7.14 Shipping & Delivery

- Shipping zones by postcode prefix (UK) with tiered service options (standard, express, same-day in selected areas).
- Carrier integrations (v1): **Royal Mail, DPD, Evri, ParcelForce, Yodel** (at minimum two at launch).
- Cold-chain carriers: dedicated couriers for frozen/chilled where standard carriers aren't viable.
- **Delivery slot booking** — calendar UI with capacity per slot per warehouse; slot caps prevent over-promising; capacity is configurable.
- Estimated delivery date calculation: cutoff times + carrier SLA + warehouse processing time.
- Free-shipping rules: spend threshold, promo-driven, or tier-based.
- Click-and-collect (optional, v1.1) for warehouses with retail front.

7.15 Tax

- UK VAT engine **MUST** handle standard (20%), reduced (5%), and zero-rated food products (most groceries are zero-rated in UK; some, e.g., confectionery and certain prepared foods, are standard-rated). Tax classes per product.
- Tax **MUST** be calculated server-side at checkout against the customer's delivery address.
- Tax-inclusive vs tax-exclusive display configurable; UK default is **inclusive**.
- Cross-border readiness: architecture supports tax rules per destination country and product tax class. EU IOSS, customs duties, and post-£0 customs threshold (effective 2026) **MUST** be accommodated when international shipping is enabled.
- All tax calculations **MUST** be logged with the rule applied for audit/forensic review.

7.16 Cargo / Freight Service

A distinct module for Ereko's parallel service (shipping packages to West Africa).

- Public inquiry form on storefront.
- Backoffice case management: queue, statuses, customer correspondence thread, quote generation, document attachments (commercial invoice, packing list).
- Status communication: customer receives status updates via email/SMS.
- Pricing engine (v1.1): semi-automated quoting based on weight/volume and destination.

7.17 Customer Support Workspace

- Unified inbox: tickets from email, contact form, WhatsApp, and in-app chat (optional).
- Each ticket MUST be linkable to a customer and order.
- Macros for common responses.
- Internal notes vs customer-visible replies.
- SLA tracking with escalation rules.
- Optional integration: Zendesk / Intercom / Front (decision in ADR-002).

7.18 Reporting & Analytics

7.18.1 Operational Reports

- Sales: daily/weekly/monthly, by channel, by category, by warehouse.
- Inventory: stock-on-hand, days-of-cover, reorder suggestions, dead stock.
- Fulfillment: cycle time, SLA breaches, error rate, picker productivity.
- Returns: rate per SKU/category, top reasons, financial impact.

7.18.2 Customer Analytics

- Cohort retention (acquisition month × months since).
- CLV / LTV per cohort and per acquisition channel.
- AOV, purchase frequency, repeat rate.
- Segment health: dormancy, win-back response.

7.18.3 Acquisition & Funnel

- Funnel: visit → product view → add to cart → checkout start → payment → success.
- Search analytics: top queries, zero-result queries, query → conversion.
- Promotion performance: redemption count, revenue lift, cannibalization.

All reports MUST be exportable (CSV, Excel) and schedulable (email digest).

8. Frontend Architecture

8.1 Tech Stack

- **Framework:** Next.js 15 (App Router) on Vercel.
- **Language:** TypeScript (strict mode, `noUncheckedIndexedAccess`, `exactOptionalPropertyTypes`).
- **UI:** React Server Components (RSC) by default; Client Components only where interactivity demands it.
- **Styling:** Tailwind CSS with a design system layer (tokens, primitives, components) built on Radix UI primitives for accessibility.
- **State (client):** TanStack Query for server-state caching; Zustand for transient UI state (cart drawer, modals).
- **Forms:** React Hook Form + Zod for schema validation (shared schemas between client and server).
- **Animations:** Framer Motion sparingly; respect `prefers-reduced-motion`.
- **PWA:** Manifest + service worker for offline product browsing, "Add to Home Screen," and (v1.1) push notifications.
- **Internationalization:** `next-intl` configured for `en-GB` at launch; locale-keyed routes (`/en-gb/...`) so future locales add without refactor.

8.2 Rendering Strategy

Surface	Strategy
Home, category, recipe, content pages	ISR (Incremental Static Regeneration) with <code>revalidate=300</code> + on-demand revalidation on publish
Product detail	ISR with on-demand revalidation on stock/price change
Search results	Server-rendered with client-side hydration
Cart, checkout, account	Dynamic (server-rendered per request, no caching)
Admin	Dynamic, client-heavy with server actions for mutations

8.3 Component Architecture

- **Atoms:** Button, Input, Badge, Icon, Skeleton.
- **Molecules:** ProductCard, PriceDisplay, StockBadge, AddToCartControl.
- **Organisms:** ProductGrid, CategoryNav, CartDrawer, CheckoutSummary, AdminTable.
- **Templates:** PDP layout, Category layout, Checkout layout, Admin layout.
- **Pages:** Composed templates with data-fetching at the page level (RSC).

Storybook MUST be maintained for all atoms and molecules; visual regression tests via Chromatic.

8.4 Performance Budgets (Enforced in CI)

Metric	Target
LCP (mobile, 4G)	≤ 2.0 s
INP	≤ 200 ms
CLS	≤ 0.1
TTFB	≤ 600 ms (cached page); ≤ 1.2 s (dynamic)
JS bundle (home, gzipped)	≤ 180 KB
JS bundle (PDP, gzipped)	≤ 220 KB
Image weight per viewport	≤ 1.5 MB
Lighthouse Performance (mobile)	≥ 85

CI MUST run Lighthouse CI on PRs targeting `main` ; regressions > 5 points block merge.

8.5 Accessibility (WCAG 2.2 AA)

Every page MUST meet WCAG 2.2 AA. Specific commitments:

- All interactive elements reachable via keyboard with visible focus.
- All form inputs have programmatic labels; errors associated via `aria-describedby` .
- Color contrast ≥ 4.5:1 for body text, ≥ 3:1 for large text/UI components.
- All images have meaningful `alt` text (decorative images use `alt=""`).
- Modals trap focus and restore on close.
- ARIA landmarks: `<header>` , `<nav>` , `<main>` , `<footer>` properly structured.
- Live regions for dynamic updates (cart count, error toasts).
- Form errors are textual, not color-only.
- Skip-to-content link on every page.
- Tested with axe-core in CI, NVDA + VoiceOver manually before each release.

9. Backend Architecture

9.1 Tech Stack

- **Database & Auth:** Supabase (Postgres 15+, Auth, Storage, Realtime, Edge Functions).
- **API layer:** Next.js Route Handlers + Server Actions, with selected Edge Functions on Supabase for low-latency, region-pinned operations.
- **Async / Queues:** Upstash QStash for delayed jobs; Supabase realtime for pub/sub.
- **Caching:** Upstash Redis for hot reads (product detail, category trees, session-bound carts).

- **Payments:** Stripe (Payment Intents, Setup Intents, Customer, Subscriptions for v1.1, Connect for v2 multi-vendor).
- **Search:** Algolia (managed, cost permitting) **or** self-hosted Meilisearch on Fly.io/Render. Decision in ADR-001.
- **Storage:** Supabase Storage for user-uploaded content; Cloudflare Images for product imagery transformation.
- **Email:** Postmark for transactional, Resend or Klaviyo for marketing.
- **SMS / WhatsApp:** Twilio.
- **Address validation:** Loqate (getAddress.io is acceptable budget alternative).

9.2 Service Boundaries (Logical, Not Microservices)

This is a modular monolith, not a microservice swarm. Each module has a clear API surface, owns its tables, and emits events. Module boundaries:

1. Identity & Auth
2. Catalog & PIM
3. Pricing & Promotions
4. Cart
5. Checkout
6. Payments
7. Orders
8. Inventory & Warehouse
9. Shipping & Tax
10. Loyalty
11. Notifications
12. Returns & Refunds
13. Content & Recipes
14. Reporting & Analytics
15. Admin & Audit
16. Cargo / Freight Service

Each module exposes a typed service layer (`/services/<module>/index.ts`) consumed by route handlers and server actions. No module reaches into another's tables directly.

9.3 API Design

9.3.1 Conventions

- All APIs MUST be **versioned** via URL prefix: `/api/v1/...`.
- Request/response bodies MUST be JSON with `Content-Type: application/json`.
- Error responses MUST follow RFC 7807 (Problem Details): `json { "type": "https://ereko.market/errors/insufficient-stock", "title": "Insufficient Stock",`

```
"status": 409, "detail": "SKU EGUSI-1KG only has 2 units available; requested 5.", "instance": "/api/v1/cart/items/...", "trace_id": "01HJ..." }
```

- All mutating endpoints MUST accept `Idempotency-Key` header and store keys for 24 hours.
- Pagination: cursor-based (`?cursor=...&limit=...`) preferred over offset for large lists.
- Filtering: query params with prefix (`?filter[status]=paid&filter[created_at][gte]=...`).
- Rate limits: per-endpoint, per-user/IP; documented in API reference.
- Auth: Bearer JWT (Supabase session) for customer/staff; HMAC-signed for service-to-service.

9.3.2 Canonical Endpoints (excerpt)

Endpoint	Method	Purpose
<code>/api/v1/products</code>	GET	List products with filters
<code>/api/v1/products/{slug}</code>	GET	Product detail
<code>/api/v1/search</code>	GET	Search products + recipes
<code>/api/v1/cart</code>	GET/POST	Get or create cart
<code>/api/v1/cart/items</code>	POST	Add item
<code>/api/v1/cart/items/{id}</code>	PATCH/DELETE	Update/remove item
<code>/api/v1/checkout/start</code>	POST	Initiate checkout, reserve stock
<code>/api/v1/checkout/payment-intent</code>	POST	Create payment intent (idempotent)
<code>/api/v1/checkout/confirm</code>	POST	Finalize after payment
<code>/api/v1/orders</code>	GET	Customer order list (auth)
<code>/api/v1/orders/{id}</code>	GET	Order detail
<code>/api/v1/orders/{id}/returns</code>	POST	Initiate RMA
<code>/api/v1/loyalty/account</code>	GET	Loyalty balance
<code>/api/v1/loyalty/redeem</code>	POST	Apply redemption to cart
<code>/api/v1/admin/...</code>	*	Admin endpoints (RBAC-protected)
<code>/api/v1/webhooks/stripe</code>	POST	Stripe webhook receiver
<code>/api/v1/webhooks/{carrier}</code>	POST	Carrier shipment webhook

9.3.3 OpenAPI

The full API MUST be specified in an OpenAPI 3.1 document, generated from Zod schemas via `zod-openapi`. The spec is the single source of truth for client SDK generation, contract tests, and external integrators.

9.4 Background Jobs

- **Stock reservation expiry** — every 60s, release expired reservations.

- **Webhook retry processor** — retries failed downstream notifications with exponential backoff.
- **Abandoned cart sender** — runs hourly, evaluates eligibility.
- **Order auto-cancellation** — cancels orders stuck in `PENDING_PAYMENT` beyond TTL.
- **Stock alert generator** — every 15 minutes, generates low-stock alerts.
- **Daily reports** — sales digest, inventory snapshot, ops digest.
- **Loyalty tier recalculation** — daily.
- **Recommendation refresh** — daily, regenerates personalization features.

All jobs MUST be: - Idempotent. - Observable (logged, traced). - Retryable with exponential backoff and a dead-letter queue. - Bounded in runtime; long-running jobs split into batches.

10. Data Model

10.1 Core Tables (selected)

The following is a non-exhaustive but representative schema. Full ER diagram is maintained in `/docs/erd.dbml`.

```

-- Users (Supabase auth.users is the source of truth; app extends with profile)
profiles (
  id uuid pk references auth.users(id) on delete cascade,
  email citext unique not null,
  phone text,
  first_name text,
  last_name text,
  preferred_locale text default 'en-GB',
  marketing_email_opt_in boolean default false,
  marketing_sms_opt_in boolean default false,
  loyalty_tier text default 'member',
  loyalty_points_balance integer default 0,
  created_at timestampz default now(),
  updated_at timestampz default now()
);

roles (id uuid pk, name text unique, description text);
role_permissions (role_id uuid fk, permission text, scope jsonb);
user_roles (user_id uuid fk, role_id uuid fk, granted_by uuid, granted_at timestampz, expires_at timestampz);

addresses (
  id uuid pk,
  user_id uuid fk,
  type text check (type in ('shipping', 'billing', 'both')),
  line1 text, line2 text, city text, region text,
  postcode text, country_code text,
  is_default boolean default false,
  validated boolean default false,
  created_at timestampz default now()
);

-- Catalog
categories (id uuid pk, parent_id uuid, slug text unique, name text, position int, meta jsonb);
products (
  id uuid pk,
  slug text unique not null,
  title text not null,
  brand text,
  origin_country text,
  description_short text,
  description_long text,
  storage_type text check (storage_type in ('ambient', 'chilled', 'frozen')),
  is_published boolean default false,
  version int default 1,
  seo jsonb,
  cultural_meta jsonb, -- regional cuisine, local names, pairings
  created_at timestampz, updated_at timestampz
);
product_variants (
  id uuid pk,
  product_id uuid fk,
  sku text unique not null,
  ean text,
  weight_grams int,
  price_amount_minor int not null, -- pence
  currency text default 'GBP',
  compare_at_amount_minor int,
  tax_class_id uuid,
  is_active boolean default true
);
product_images (id uuid pk, product_id uuid fk, url text, alt text, position int);
product_categories (product_id uuid fk, category_id uuid fk, primary key (product_id, category_id));
product_dietary_tags (product_id uuid fk, tag text, primary key (product_id, tag));
product_allergens (product_id uuid fk, allergen text);

```

```

-- Inventory
warehouses (id uuid pk, code text unique, name text, address jsonb, is_active boolean, capabilities jsonb);
warehouse_stock (
  id uuid pk,
  warehouse_id uuid fk,
  variant_id uuid fk,
  on_hand int not null default 0,
  reserved int not null default 0,
  damaged int not null default 0,
  safety_stock int default 0,
  unique (warehouse_id, variant_id)
);
inventory_movements (
  id uuid pk,
  variant_id uuid fk,
  warehouse_id uuid fk,
  type text check (type in ('receipt', 'sale', 'return', 'transfer_in', 'transfer_out', 'adjustment', 'other')),
  quantity int not null,
  reference_type text, -- 'order', 'rma', 'transfer', 'manual'
  reference_id uuid,
  actor_id uuid,
  reason_code text,
  created_at timestamptz default now()
);

-- Cart & Checkout
carts (id uuid pk, user_id uuid, anonymous_token text, currency text, created_at, expires_at);
cart_items (id uuid pk, cart_id uuid fk, variant_id uuid fk, quantity int, unit_price_minor int, unit_price_major int);

stock_reservations (
  id uuid pk,
  variant_id uuid fk,
  warehouse_id uuid fk,
  cart_id uuid,
  order_id uuid,
  quantity int,
  expires_at timestamptz,
  released_at timestamptz
);

-- Orders
orders (
  id uuid pk,
  order_number text unique not null, -- human-friendly, e.g., EREKO-2026-000123
  user_id uuid,
  email citext not null,
  status text not null,
  currency text default 'GBP',
  subtotal_minor int, discount_minor int, shipping_minor int, tax_minor int, total_minor int,
  shipping_address jsonb, billing_address jsonb,
  delivery_slot jsonb,
  notes_customer text, notes_internal text,
  loyalty_points_earned int default 0, loyalty_points_redeemed int default 0,
  placed_at timestamptz default now(),
  paid_at timestamptz, cancelled_at timestamptz, fulfilled_at timestamptz
);
order_items (id uuid pk, order_id uuid fk, variant_id uuid fk, sku text, title text, quantity int, unit_price_major int, unit_price_minor int);
order_events (id uuid pk, order_id uuid fk, event_type text, payload jsonb, actor_id uuid, created_at timestamptz);

shipments (id uuid pk, order_id uuid fk, warehouse_id uuid fk, carrier text, service text, tracking_id text);
shipment_items (shipment_id uuid fk, order_item_id uuid fk, quantity int);

returns (id uuid pk, order_id uuid fk, status text, reason_code text, customer_note text, photos jsonb);
return_items (return_id uuid fk, order_item_id uuid fk, quantity int, restock boolean);

-- Payments

```

```

payments (
  id uuid pk,
  order_id uuid fk,
  stripe_payment_intent_id text unique,
  amount_minor int, currency text,
  status text, -- requires_action, succeeded, failed, refunded
  raw_event jsonb,
  created_at timestampz default now()
);
refunds (id uuid pk, payment_id uuid fk, amount_minor int, reason text, status text, stripe_refund_id text);

-- Promotions
promotions (id uuid pk, code text unique, name text, type text, value jsonb, rules jsonb, starts_at timestampz, ends_at timestampz);
coupon_redemptions (id uuid pk, promotion_id uuid fk, user_id uuid, order_id uuid, redeemed_at timestampz);

-- Loyalty
loyalty_transactions (id uuid pk, user_id uuid fk, type text, points int, reference_type text, reference_id uuid, created_at timestampz);
referrals (id uuid pk, referrer_id uuid, referee_email citext, code text, status text, reward_created_at timestampz);

-- Tax & Shipping
tax_classes (id uuid pk, name text);
tax_rules (id uuid pk, tax_class_id uuid fk, country_code text, region text, rate_bps int);
shipping_zones (id uuid pk, name text, postcode_patterns text[]);
shipping_rates (id uuid pk, zone_id uuid fk, service_name text, carrier text, price_minor int, price_major int, created_at timestampz);
delivery_slots (id uuid pk, warehouse_id uuid fk, slot_date date, slot_start time, slot_end time);

-- Content
recipes (id uuid pk, slug text unique, title text, body jsonb, hero_image text, cook_time_min int, cook_time_max int);
recipe_ingredients (recipe_id uuid fk, variant_id uuid fk, quantity_text text);
content_pages (id uuid pk, slug text unique, title text, body jsonb, is_published boolean);

-- Admin
audit_logs (
  id uuid pk,
  actor_id uuid,
  actor_role text,
  action text,
  resource_type text,
  resource_id uuid,
  before jsonb,
  after jsonb,
  reason text,
  ip inet,
  user_agent text,
  created_at timestampz default now()
);
notifications_outbound (id uuid pk, channel text, recipient text, template text, payload jsonb, status text, created_at timestampz);
idempotency_events (key text pk, route text, request_hash text, response_status int, response_body jsonb, created_at timestampz);
feature_flags (key text pk, value jsonb, updated_by uuid, updated_at timestampz);

```

10.2 Schema Conventions

- Money stored as **integer minor units** (pence). Currency code on every monetary record.
- Timestamps are `timestampz`, always UTC.
- Soft-deletes via `deleted_at` only where required; otherwise hard delete + audit log.
- `created_at` / `updated_at` triggers on every table.
- UUIDs (v7 preferred for time-ordering) for all primary keys.
- All FKs have `ON DELETE` policies explicitly chosen (no implicit cascades).

10.3 Row-Level Security (RLS)

RLS is **enabled on every customer-facing table**. Sample policies:

```
-- Customers see only their own orders
create policy "orders_owner_read" on orders for select
  using (auth.uid() = user_id);

-- Staff with orders:read see orders for warehouses they're assigned to
create policy "orders_staff_read" on orders for select
  using (
    exists (
      select 1 from user_roles ur
      join role_permissions rp on rp.role_id = ur.role_id
      where ur.user_id = auth.uid()
      and rp.permission = 'orders:read'
    )
  );

-- Inventory mutations restricted to warehouse operators
create policy "inventory_write_warehouse_staff" on inventory_movements for insert
  with check (
    auth.uid() in (
      select user_id from warehouse_assignments
      where warehouse_id = inventory_movements.warehouse_id
    )
  );
```

RLS policies are version-controlled in Supabase migrations. Every policy **MUST** have a test (positive + negative) in the test suite.

10.4 Indexing & Performance

- BTree indexes on every FK.
- Composite indexes on hot query paths (e.g., `(order_status, placed_at)` for the admin orders queue).
- GIN indexes for JSONB fields queried by key.
- Materialized views for heavy reporting (refreshed nightly).
- Partitioning for `order_events` and `audit_logs` by month once volume warrants.

11. Security

Security is engineered, not bolted on. The platform aligns to **OWASP ASVS Level 2**, **PCI DSS v4.0.1 (SAQ A)** by virtue of using Stripe-hosted card collection, and **UK GDPR / DPA 2018**.

11.1 Authentication

- Supabase Auth with email/password, magic link, and OAuth (Google, Apple) options.
- Argon2id password hashing (Supabase default).

- Password policy: minimum 10 characters, NIST 800-63B style (no forced rotation, breach-check via HIBP API).
- MFA REQUIRED for all staff roles (TOTP at minimum; WebAuthn preferred).
- Session JWTs short-lived (1 hour) with refresh tokens; refresh tokens rotated.
- Step-up auth for sensitive operations (refunds, role changes, secret rotation).

11.2 Authorization

- RBAC enforced at three layers (UI, API, RLS).
- Default-deny: any route without explicit permission check rejects.
- Service accounts have separate roles and rotate keys quarterly.

11.3 Application Security

- All inputs validated against Zod schemas at the API boundary.
- Output encoding by default (React escapes); explicit `dangerouslySetInnerHTML` audited and minimized.
- CSRF protection via SameSite=strict cookies + token for mutating non-auth endpoints.
- CSP with strict directives; nonces for inline scripts.
- HSTS preload, Referrer-Policy, X-Frame-Options DENY, Permissions-Policy.
- Rate limiting (Upstash) per route and per user/IP, with bot detection (Cloudflare Bot Management).
- SQL injection prevented by Postgres parameterized queries (no raw concatenation).
- Dependency scanning (Snyk or GitHub Advanced Security) in CI; weekly SBOM generation.

11.4 Payments Security

- Card data **never touches our servers**. Stripe Elements / Payment Element used for collection.
- Webhooks signature-verified with Stripe signing secret on every request.
- Idempotency keys on all create/refund operations.
- 3-D Secure 2 always enabled; Radar rules for fraud screening.
- Refund authorization gated by RBAC + step-up auth.

11.5 Data Protection (GDPR / UK-DPA)

- **Lawful basis** documented per data category (consent for marketing, contract for transactional, legitimate interest for fraud prevention).
- **Consent management** for cookies and marketing comms with audit log of consent changes.
- **Subject access**: customers can export their data (JSON) from the account portal.
- **Right to erasure**: account deletion request creates an erasure case; data anonymized in transactional records (orders retained for tax/legal compliance with PII redacted); full processing within 30 days.
- **Data minimization**: only collect what's needed; payment details tokenized.

- **Retention schedule** documented per table (e.g., audit logs 6 years, marketing analytics 24 months unless consent renewed).
- **DPA** in place with every processor; sub-processor list published.
- **Cookie banner** compliant with PECR + GDPR; granular consent (strictly necessary, analytics, marketing, personalization); rejected by default until consented.
- **DPIA** completed for high-risk processing (loyalty profiling, recommendation engine).
- **Breach response plan**: 72-hour notification to ICO, documented runbook.

11.6 Infrastructure Security

- Secrets in Vercel environment / Supabase Vault / 1Password; never in code.
- Production access via SSO + MFA; least privilege.
- Database backups: automated daily + PITR (7-day window); restore drills quarterly.
- WAF: Cloudflare with managed rule sets + custom rules for known attack patterns.
- DDoS mitigation: Cloudflare.
- Logging: structured logs to centralized store; PII redacted at write.
- Tamper-evident audit log (append-only, optionally signed with hash chain).

11.7 Threat Model Highlights

Threat	Mitigation
Account takeover via credential stuffing	HIBP check, rate limit, MFA for staff, anomaly detection for customers
Inventory oversell from race conditions	Postgres <code>SELECT FOR UPDATE</code> + reservation table
Coupon abuse / referral farming	Velocity rules, device fingerprinting, manual review queue
Refund fraud	Approval thresholds, step-up auth, audit log
Webhook replay	Idempotency events table, signature verification, timestamp tolerance window
Bot scraping / scalping	Cloudflare Bot Management, rate limits, hCaptcha on sensitive endpoints
Insider threat	RBAC, audit log, separation of duties, deputization with expiry

12. Accessibility (WCAG 2.2 AA) — Detailed Requirements

WCAG 2.2 AA is treated as a hard requirement reinforced by ongoing audits.

Area	Requirement
Forms	Programmatic labels, error text + <code>aria-describedby</code> , autocomplete attributes (<code>autocomplete="postal-code"</code> etc.), <code>inputmode</code> for mobile keyboards

Area	Requirement
Navigation	Skip link, landmarks, focus visible (2.4.11), focus not obscured (2.4.11), consistent help (3.2.6)
Images	Meaningful alt; decorative <code>alt=""</code> ; product images describe origin and packaging
Color	Contrast $\geq 4.5:1$ body, $\geq 3:1$ large/UI, never color-only meaning
Motion	<code>prefers-reduced-motion</code> honored; auto-play disabled
Modals & Drawers	Focus trap; ESC closes; focus returned on close
Tables (admin)	<code><th scope></code> , caption, sortable headers announce sort state
Tooltips	Hover + focus + dismissable (1.4.13)
Errors	Inline + summary at top, focus moves to first error
Drag targets	Drag-and-drop has keyboard alternative (2.5.7)
Authentication	No cognitive function test required to log in (3.3.8)

Audit cadence: - **Per PR:** axe-core checks in CI; manual keyboard test on changed UI. - **Per release:** Lighthouse a11y ≥ 95 ; NVDA + VoiceOver spot checks of changed flows. - **Quarterly:** Full third-party WCAG 2.2 AA audit.

An **Accessibility Statement** page MUST be published, listing conformance level, known issues, and contact for accessibility feedback.

13. Performance, Caching, and Scaling

13.1 Caching Strategy

Cache	Use	TTL
Cloudflare edge (page)	Public pages (home, category, PDP)	5 min, on-demand invalidation on publish
Cloudflare edge (image)	All product images	1 year (versioned URLs)
Upstash Redis	Hot product data, category trees, exchange rates, session carts	5 min (data), 24h (carts)
Postgres materialized views	Reports, leaderboards	nightly refresh
Search index	Algolia/Meilisearch	live (webhook-driven)

Cache invalidation: - Catalog publish → on-demand revalidation of affected pages + search reindex. - Stock change → real-time push to PDP via Supabase Realtime if PDP is open. - Promotion launch → bust home + category pages.

13.2 Scaling Posture

- Vercel autoscales Next.js.
- Supabase: start on a small instance; scale vertically before horizontal sharding becomes necessary; use read replicas for reporting once writes saturate primary.
- Stateless application; all state in Postgres / Redis / Storage.
- Background jobs scale independently via Upstash QStash concurrency settings.

13.3 Capacity Planning

Targets at year 1: - 50k MAU, 5k orders/month at peak, 500 concurrent sessions at peak (Black Friday / Christmas). - 99.95% availability SLO on storefront read paths; 99.9% on checkout and payment paths (excludes Stripe upstream).

14. Observability

14.1 Telemetry Stack

- **Logs:** Structured JSON via Pino; shipped to Datadog or Better Stack.
- **Metrics:** OpenTelemetry → Datadog/Grafana Cloud; RED metrics (Rate, Errors, Duration) per route.
- **Traces:** OpenTelemetry distributed tracing across Next.js, Supabase Edge Functions, and outbound calls.
- **Error tracking:** Sentry (frontend + backend) with source maps.
- **Product analytics:** PostHog or Mixpanel for funnel, retention, feature usage.
- **Real-user monitoring (RUM):** Sentry Performance or Vercel Speed Insights.

14.2 Required Dashboards

1. **Commerce health** — orders/hour, conversion, AOV, payment failures, top error pages.
2. **Fulfillment health** — orders in each state, oldest unallocated, SLA breaches.
3. **Inventory health** — items below safety stock, dead stock, days-of-cover.
4. **Webhook health** — Stripe + carriers + ESP success/fail rates, retry queue depth.
5. **System health** — p95 latency, error rate, DB CPU, cache hit rate.
6. **Marketing** — campaign-driven revenue, segment activity, abandoned-cart recovery rate.

14.3 Alerts

Severity-tiered, paging vs ticketing:

Severity	Examples	Response
SEV-1 (page)	Checkout error rate > 5% for 5 min; Stripe webhook failure > 10/min; DB down	On-call paged 24/7

Severity	Examples	Response
SEV-2 (alert)	Payment p95 > 5s; abandoned-cart job stalled	Business hours
SEV-3 (notice)	Low stock without replenishment; deliverability dip	Daily digest

Every alert links to a runbook.

15. QA & Testing Strategy

15.1 Test Pyramid

- **Unit tests** (Vitest): Pure business logic — pricing, tax, promotions, allocation algorithms, points calculation. Target coverage $\geq 85\%$ on services modules.
- **Integration tests** (Vitest + Supabase test container): API routes hitting a real Postgres test database, including RLS policy tests.
- **Contract tests**: Webhook payload contracts validated against Zod schemas; Stripe-CLI replay tests.
- **End-to-end tests** (Playwright): Critical journeys — browse, add-to-cart, checkout, account, admin core flows, return flow.
- **Visual regression** (Chromatic): Atoms, molecules, and key page snapshots.
- **Accessibility tests** (axe-core): In Playwright suite + Storybook.
- **Performance tests**: Lighthouse CI on PRs; k6 load tests against staging quarterly.
- **Security tests**: SAST (GitHub CodeQL), dependency scanning, secret scanning, ZAP DAST in staging.

15.2 Test Data

- Seed scripts for development (representative African product catalog, multiple warehouses, sample orders in every state).
- Anonymized snapshots of production for staging (PII scrubbed).
- Test customers with predictable cards (Stripe test card numbers).

15.3 Release Gates

A PR may merge only if: 1. All tests pass. 2. Lighthouse CI within budget (or budget waiver approved). 3. axe-core has zero serious/critical issues on changed pages. 4. Type checks pass with strict mode. 5. ESLint passes. 6. Bundle size within budget. 7. At least one human approval. 8. Migration plan reviewed (if DB changes).

16. DevOps, CI/CD, and Environments

16.1 Environments

Env	Purpose	Data	Access
Local	Dev	Seeded	Engineer
Preview	Per-PR	Seeded ephemeral	Engineer + reviewers
Staging	Pre-prod	Sanitized prod snapshot	Engineering + QA + business UAT
Production	Live	Live	Restricted; deployment via CI

16.2 CI/CD

- GitHub Actions on every push: lint, type, unit, integration.
- On PR to `main`: e2e on preview deployment, Lighthouse, axe.
- On merge to `main`: deploy to staging automatically; manual promotion to production with change-record.
- Database migrations applied via Supabase CLI with `--linked` to environment; migrations are forward-only; destructive changes require two reviewers.
- Feature flags (e.g., Flagsmith / GrowthBook / homegrown table) for risk gating new features behind flags before public release.

16.3 Rollback

- Vercel instant rollback to last known good.
- Database migrations are backward-compatible (expand → contract pattern).
- Feature flags allow instant kill-switch for problematic features without redeploy.

17. Integrations

Each integration has a contract, retry strategy, failure mode, and manual fallback.

Integration	Purpose	Failure Mode	Fallback
Stripe	Payments, refunds, disputes	Card collection blocked	Maintenance banner; queued webhooks resume on recovery
Postmark	Transactional email	Outbound queue grows	Auto-failover to backup ESP (SendGrid) if configured
Twilio	SMS / WhatsApp	Drop SMS, retain email	Email-only updates with banner
	Labels, tracking	Manual label printing	Ops uses carrier portal manually

Integration	Purpose	Failure Mode	Fallback
Carrier APIs (Royal Mail, DPD, Evri)			
Loqate / getAddress.io	Address validation	Manual address entry only	Skip validation, mark unvalidated
Algolia / Meilisearch	Search	Fall back to Postgres full-text search	Reduced relevance, no typo tolerance
Cloudflare Images	Image transformations	Serve from Supabase Storage directly	Larger images, slower CWV
Klaviyo / Resend	Marketing email	Pause campaigns	Resume on recovery
PostHog / Mixpanel	Analytics	Event queue locally buffered	None required (non-critical)
Xero / QuickBooks export	Accounting	CSV manual export	Manual import

All integrations MUST: - Verify webhook signatures. - Use idempotency keys. - Retry with exponential backoff (max 6 attempts, jitter). - Surface failures on the Webhook Health dashboard. - Be configurable via admin Integrations panel (API keys, toggles).

18. AI & Personalization

A pragmatic, opt-in approach to AI.

18.1 v1 Capabilities

- **Personalized product rails** on home and PDP (similar items, frequently bought together, complete the recipe) using collaborative filtering + content-based features (cuisine, origin, dietary tags).
- **Search relevance learning** via Algolia/Meilisearch click models.
- **AI-assisted PIM** — descriptions, alt-text suggestions, category recommendations during product creation; staff retains final say.
- **Recipe-to-cart** — recipe page can add all linked SKUs in one click.
- **Customer support assistant (internal)** — agents see suggested replies and similar past cases; never auto-replies to customers without human approval.

18.2 v1.1+ Capabilities

- AI chat for storefront product discovery ("I want to make jollof rice for 6 people, what do I need?") with explicit "human handoff" path.
- Demand forecasting per SKU per warehouse to drive reorder suggestions.
- Substitution recommender that learns from accept/reject patterns.

18.3 Governance

- All AI features **MUST** be labeled as such ("AI-generated description").
 - Customer-facing AI must not invent product attributes; PIM AI is a *suggestion* engine, never a publishing engine.
 - Personalization respects consent: customers without analytics consent receive non-personalized, deterministic merchandising.
-

19. Internationalization & Localization Readiness

v1 ships **en-GB** only. Architecture must support adding locales without refactor.

- All user-facing strings via `next-intl` message catalogs.
 - Date, currency, number formatting via `Intl` APIs.
 - URL strategy: `/en-gb/...` (locale always in URL).
 - Product translations: separate `product_translations` table keyed on `(product_id, locale)` for v2; v1 uses single-locale fields with the locale tagged.
 - Currency: GBP only at launch; data model supports multiple currencies; FX rates table prepared for v2.
 - Tax rules already per-country.
-

20. Content & Marketing Operations

20.1 CMS

- Product, category, and content pages managed in the admin CMS.
- Recipes are first-class content with SKU links.
- Banners and homepage rails managed by Marketing (Campaign Builder): schedule, target segments, A/B test.
- Email templates managed in admin with preview and test-send.
- All content versioned; rollback to previous version supported.

20.2 SEO

- Server-rendered pages with semantic HTML.
- Auto-generated `sitemap.xml` updated on publish.
- `Robots.txt` curated.
- Structured data on PDP (Product, Offer, Review/AggregateRating), Recipe pages, Breadcrumbs, Organization, LocalBusiness.
- Canonical URLs to prevent duplicate content from facets.

- Open Graph + Twitter Card on every page.
- Hreflang-ready for future locales.

20.3 Marketing Channels

- Email (transactional + lifecycle + marketing).
- SMS / WhatsApp for opted-in segments.
- Affiliate / influencer codes via the promotion engine.
- Referral program in loyalty.
- Social commerce (Meta catalog feed, TikTok shop feed) export in v1.1.

21. Operational Runbooks (Required)

Each runbook is a short, executable doc kept in `/docs/runbooks/`.

1. **Payment provider outage** — switch to maintenance banner, queue intents, replay on recovery.
2. **Inventory desync (real vs system)** — cycle count workflow, lock SKU, replay movements.
3. **Carrier outage** — switch primary carrier, flag affected orders.
4. **Webhook backlog** — drain queue, identify root cause, replay safely.
5. **Database failover** — Supabase PITR procedure, communications template.
6. **DDoS / traffic spike** — Cloudflare protections, rate-limit toggles.
7. **Data subject request (GDPR)** — SLA, export procedure, erasure procedure.
8. **Security incident** — triage, contain, communicate, post-mortem.
9. **Refund fraud spike** — tighten Radar rules, escalation chain.
10. **Black Friday / Christmas load** — pre-event checklist, hot-fixes-only freeze, scale-up plan.

22. Definition of Done

A unit of work is "Done" only when:

1. Code merged to `main` via reviewed PR with all CI gates green.
2. Tests added (unit + integration + e2e where applicable) and passing.
3. Accessibility verified on affected UI (axe + manual keyboard + screen reader on critical paths).
4. Performance budget maintained (Lighthouse CI green).
5. Telemetry added (logs, metrics, traces) for new behavior.
6. Feature flags configured if release-gated.
7. Documentation updated (`/docs` , API reference, runbook if operational).
8. Security review checklist completed (RLS, input validation, secrets, dependencies).
9. Migrations applied to staging and verified against sanitized snapshot.

10. Stakeholder UAT signed off (for customer-facing or operational changes).
 11. Rollback plan documented.
 12. Deployed to production and verified live (smoke checks + key dashboards green for 30 minutes post-deploy).
-

23. Acceptance Criteria — Platform Level

The platform is considered launch-ready when **all** of the following are demonstrably true:

Commerce

- A guest customer can browse, search, filter, add to cart, and complete checkout end-to-end on mobile in ≤ 6 minutes elapsed.
- A returning customer can complete checkout in ≤ 90 seconds using Apple Pay.
- Stripe webhook successfully transitions 99.9% of paid orders to **PAID** within 30 seconds of payment.
- Cart price-revalidation surfaces any discrepancy before customer reaches payment.
- All payment methods (Visa, MC, Amex, Apple Pay, Google Pay, Klarna, Clearpay) succeed end-to-end with 3DS where required.
- Promotions and loyalty redemptions reduce cart total correctly and are reversible on cancellation.

Operations

- Warehouse Operator can complete a 10-item pick on a tablet with barcode scanning in ≤ 5 minutes.
- Substitution policy is honored per SKU and per customer preference, with audit trail.
- Refund flow respects RBAC thresholds, requires step-up auth, and executes Stripe refund within 10 seconds.
- Return RMA can be initiated by a customer, approved by Support, and closed with refund + inventory restock without manual data entry beyond the approval click.
- Audit log captures every privileged action with full diff (before/after).

Quality

- WCAG 2.2 AA conformant on home, category, PDP, cart, checkout, account, and admin dashboard (external audit).
- All pages pass Core Web Vitals thresholds for mobile on the latest Chrome User Experience Report sample.
- p95 storefront read latency ≤ 300 ms at edge.
- 99.95% storefront availability over the launch quarter.
- Zero PII leakage in logs (verified via log scan).
- Penetration test (third-party) completed with no high/critical findings unremediated.

Operations Readiness

- All 10 core runbooks published.
 - On-call rotation staffed with paging configured.
 - Daily backup + monthly restore drill executed successfully.
 - Staff training completed on admin, warehouse, and support workflows.
-

24. Delivery Roadmap

The rebuild is organized into four phases. Each phase ends with an internal milestone demo, UAT, and security review.

Phase 1 — Foundation (Weeks 1–6)

- Repo, CI/CD, environments, design system, auth.
- Catalog (PIM-lite), category trees, search v1 (Postgres FTS).
- Cart, checkout (cards + Apple Pay + Google Pay), Stripe integration.
- Order creation + payment confirmation + basic order detail.
- Admin shell with auth, RBAC, audit log skeleton.
- Email (transactional templates: order confirmation, receipt).
- Observability baseline (logs, errors, RUM).
- Security baseline (RLS, headers, rate limits, secrets).

Exit criteria: A guest can place a paid order end-to-end against a small seeded catalog; admin can view orders.

Phase 2 — Operations (Weeks 7–12)

- Multi-warehouse inventory, reservations, allocation engine.
- Warehouse dashboard (receive, pick, pack, ship, transfer, cycle counts).
- Shipping zones, carrier integrations (Royal Mail + DPD), delivery slot booking.
- Tax engine (UK VAT, zero-rated handling).
- Returns + refunds workflow + RBAC thresholds.
- Substitution policy at SKU and customer level.
- Carrier webhooks for tracking.
- SMS + WhatsApp transactional channels.
- Klarna + Clearpay enabled.
- Algolia/Meilisearch search rollout.

Exit criteria: Operational pilot — staff fulfill 50 real orders end-to-end across two warehouses with all states and edge cases (substitution, partial ship, refund) exercised.

Phase 3 — Growth (Weeks 13–18)

- Loyalty program (points, tiers, referrals).
- Promotions engine (codes, automatic discounts, bundles, free shipping).
- Wishlist + Save for Later.
- Recipe content hub + ingredient-to-cart.
- Abandoned cart flow + lifecycle marketing flows.
- AI-personalized rails (collaborative + content-based).
- Customer segments + segment-targeted promotions.
- Reporting dashboards (sales, inventory, fulfillment, cohort/LTV).
- Cargo inquiry intake + backoffice case management.

Exit criteria: Public launch readiness — all v1 in-scope features complete, external WCAG audit passed, pen test completed.

Phase 4 — Enterprise & Scale (Weeks 19–26, post-launch)

- Subscriptions / repeat boxes.
- Gift cards.
- Advanced workflow engine for staff approvals (configurable rules).
- B2B/wholesale portal (light).
- Native PWA push notifications.
- Demand forecasting + reorder suggestions.
- Multi-currency + locale architecture activation (when first new market is targeted).
- Marketplace seller portal evaluation.

25. Open Questions & Decisions Pending (ADRs)

The following decisions must be ratified during Phase 1; each becomes an Architecture Decision Record.

ADR	Question	Default
ADR-001	Search vendor: Algolia (managed, cost) vs Meilisearch (self-hosted, control)	Meilisearch self-hosted on Render; revisit at 100k SKUs
ADR-002	Support inbox: Build vs Front/Intercom/Zendesk	Front at scale; admin-internal CRM for v1
ADR-003	Analytics: PostHog (self-host option) vs Mixpanel	PostHog Cloud (EU region)
ADR-004	ESP split: Postmark (transactional) + Klaviyo (marketing) vs single vendor	Split (best-of-breed)
ADR-005	Address validation: Loqate vs getAddress.io	Loqate (richer, multi-country future)
ADR-006		Cloudflare Images for product imagery

ADR	Question	Default
	Image hosting: Cloudflare Images vs Supabase Storage + Next/Image	
ADR-007	Carrier strategy: direct APIs vs Shippo/EasyPost aggregator	Direct for v1 (Royal Mail + DPD); aggregator at 3+ carriers
ADR-008	Loyalty: build vs Yotpo/LoyaltyLion	Build (configurable, owned data)

26. Glossary

- **AOV** — Average Order Value.
- **CLV / LTV** — Customer Lifetime Value.
- **CWV** — Core Web Vitals (LCP, INP, CLS).
- **CDE** — Cardholder Data Environment (PCI).
- **DPIA** — Data Protection Impact Assessment.
- **EAA** — European Accessibility Act.
- **HIBP** — Have I Been Pwned (breach corpus).
- **INP** — Interaction to Next Paint.
- **OWASP ASVS** — Application Security Verification Standard.
- **PIM** — Product Information Management.
- **PSD2** — Payment Services Directive 2 (SCA).
- **RMA** — Return Merchandise Authorization.
- **RBAC** — Role-Based Access Control.
- **RLS** — Row-Level Security (Postgres).
- **RSC** — React Server Components.
- **SAQ** — Self-Assessment Questionnaire (PCI).
- **SCA** — Strong Customer Authentication.
- **SLA / SLO** — Service Level Agreement / Objective.
- **SKU** — Stock Keeping Unit.
- **3DS2** — 3-D Secure version 2.

27. Final Product Positioning

When this rebuild is complete, Ereko African Market is not "an online African food shop." It is a **specialty grocery commerce platform** that happens to lead with authentic African cuisine — operationally indistinguishable from a Tier-1 grocer in checkout fluidity, inventory accuracy, accessibility, and post-purchase service, while culturally and editorially distinct in voice, merchandising, and community.

The platform's three commitments are:

1. **To customers:** A fast, accessible, trustworthy way to get the authentic ingredients of home, delivered well, every time.
2. **To operators:** Tools that make running a complex, perishable, multi-warehouse business feel orderly rather than frantic.
3. **To the business:** A platform that scales — into new categories, new regions, new lines of business — without requiring a rebuild for the fourth time.

End of Document. Version 2.0. Maintained in `/docs/PRD.md` and version-controlled with the codebase.